

Ergebnisbericht Übungsblatt 2

Dieses Übungsblatt hat das Ziel, zwei grundlegende Eigenschaften von GPUs experimentell sichtbar zu machen: erstens, dass eine GPU sehr viele Threads im Gleichschritt abarbeitet und datenabhängige Verzweigungen dadurch teuer werden (Aufgabe 1) und zweitens, dass eine GPU Speicherlatenz nicht über große Caches, sondern über massive Parallelität versteckt, weshalb das Zugriffsmuster über die tatsächlich erreichte Bandbreite entscheidet (Aufgabe 2). Für beide Aufgaben habe ich jeweils einen eigenen CUDA-Benchmark implementiert und die Ergebnisse mit dem Verhalten einer CPU verglichen. Über die geforderten Messungen hinaus habe ich zwei Erweiterungen umgesetzt: eine AVX2-Implementierung, die zeigt, dass Divergenz keine reine GPU-Eigenschaft ist, sowie eine Latenzmessung, mit der sich der nötige Parallelitätsgrad für die Bandbreitensättigung über Little's Law vorhersagen lässt.

Alle Messungen liefen auf meinem eigenen Rechner mit einer NVIDIA GeForce RTX 3070 (Ampere-Architektur, 46 Streaming Multiprocessors, maximal 48 Warps pro SM, 4 MB L2-Cache) unter Windows mit CUDA Toolkit 13 und MSVC (Build Tools 2022) als Host-Compiler. Ein Streaming Multiprocessor (SM) ist dabei die grundlegende Recheneinheit der GPU, vergleichbar mit einem Kern einer CPU, nur dass jeder SM selbst wieder viele Threads gleichzeitig verwaltet. Die theoretische Speicherbandbreite von 448 GB/s habe ich zur Laufzeit aus den Geräteeigenschaften berechnet (Speichertakt $\times$ Busbreite $\times$ 2 wegen Double Data Rate). Sämtliche Kernel-Laufzeiten habe ich mit CUDA-Events gemessen, da Kernel-Starts asynchron erfolgen und eine CPU-seitige Zeitmessung sonst nur den Startaufruf statt der eigentlichen Ausführung erfassen würde. Vor jeder Messung laufen mehrere Warmup-Durchgänge, gemittelt wird über mehrere Wiederholungen.

Aufgabe 1: SIMT-Ausführungsmodell und Warp-Divergenz

Die Aufgabe verlangt einen rechenintensiven Kernel mit minimalem Speicherbedarf: jeder Thread bearbeitet genau ein Element und führt darauf eine konfigurierbare, rein arithmetische Last aus. Anschließend soll die Problemgröße n bis in die Millionen skaliert, der Durchsatz gemessen und gezielt Warp-Divergenz eingeführt werden. Ein Warp ist dabei eine Gruppe von 32 Threads, die auf der GPU physisch im Gleichschritt ausgeführt wird. Das ist der Kern des SIMT-Modells (Single Instruction, Multiple Threads): eine Instruktion wird gleichzeitig auf 32 sogenannte Lanes angewendet. Von Divergenz spricht man, wenn Threads innerhalb desselben Warps aufgrund einer datenabhängigen Verzweigung unterschiedliche Codepfade nehmen müssten, was im Gleichschrittmmodell nicht direkt möglich ist.

Mein Kernel lädt pro Thread ein Element, führt darauf 10.000 Iterationen der Form $x = x * a + b$ aus und schreibt das Ergebnis zurück. Diese Operation ist eine FMA (Fused Multiply-Add, Multiplikation und Addition in einem Schritt), die auf GPUs als eine Instruktion mit zwei Gleitkommaoperationen (FLOPs) zählt. Zwei Implementierungsdetails waren hier wichtig: Erstens muss das Ergebnis datenabhängig bleiben und zurückgeschrieben werden, sonst optimiert der Compiler die gesamte Schleife weg und man misst nichts. Zweitens habe ich die Konstante a pro Pfad leicht unterschiedlich gewählt, damit der Compiler die Pfade nicht wieder zu einem einzigen zusammenlegen kann.

Die Divergenz erzeuge ich, wie im Aufgabenblatt vorgeschlagen, über eine Verzweigung auf Basis der Lane-Nummer: die 32 Lanes eines Warps werden über $lane \% k$ auf k verschiedene Pfade verteilt. Im Code läuft dazu eine äußere Schleife über die k Pfade und jede Lane betritt genau den einen Pfad,

der zu ihr gehört. Entscheidend für eine saubere Messung: jeder Pfad leistet exakt gleich viel Arbeit (dieselbe Iterationszahl, nur eine andere Konstante). Dadurch misst der Leistungseinbruch ausschließlich die Serialisierung der Pfade und nicht eine ungleiche Lastverteilung, die das Ergebnis verfälschen würde. Um den Divergenzgrad zu variieren nutzen wir $k = 1$, womit wir keine Divergenz haben und $k = 32$ wodurch jede Lane des Warps einen eigenen Pfad nimmt.

Als Vergleich dient eine CPU-Implementierung mit identischer Logik und identischer Verzweigung, gemessen über alle Divergenzgrade hinweg. Der Durchsatz wird jeweils in GFLOP/s angegeben, berechnet als $2 \times n \times \text{Iterationen}$ geteilt durch die Laufzeit.

Zunächst habe ich ohne Divergenz ($k = 1$) die Problemgröße n von 1.024 bis 16,7 Millionen Elemente skaliert. Das Ergebnis zeigt Abbildung 1: Bei $n = 1.024$ erreicht die GPU nur etwa 556 GFLOP/s, also weniger als 3 % ihres Maximums. Der Grund ist einfach: 1.024 Threads verteilen sich auf 46 SMs, sodass die meisten Recheneinheiten schlicht leerlaufen und zusätzlich der Overhead des Kernel-Starts ins Gewicht fällt. Mit wachsendem n steigt der Durchsatz steil an und sättigt ab etwa 260.000 Threads bei rund 20 TFLOP/s. Die GPU „läuft“ also erst ab einer Problemgröße im Bereich von mehreren hunderttausend Threads „warm“. Man muss das Gerät regelrecht mit Arbeit fluten, um es auszulasten. Genau dieser Aufwand fällt bei einer CPU-Lösung in dieser Form gar nicht erst an: ein einzelner CPU-Kern ist ab dem ersten Element voll bei der Sache.

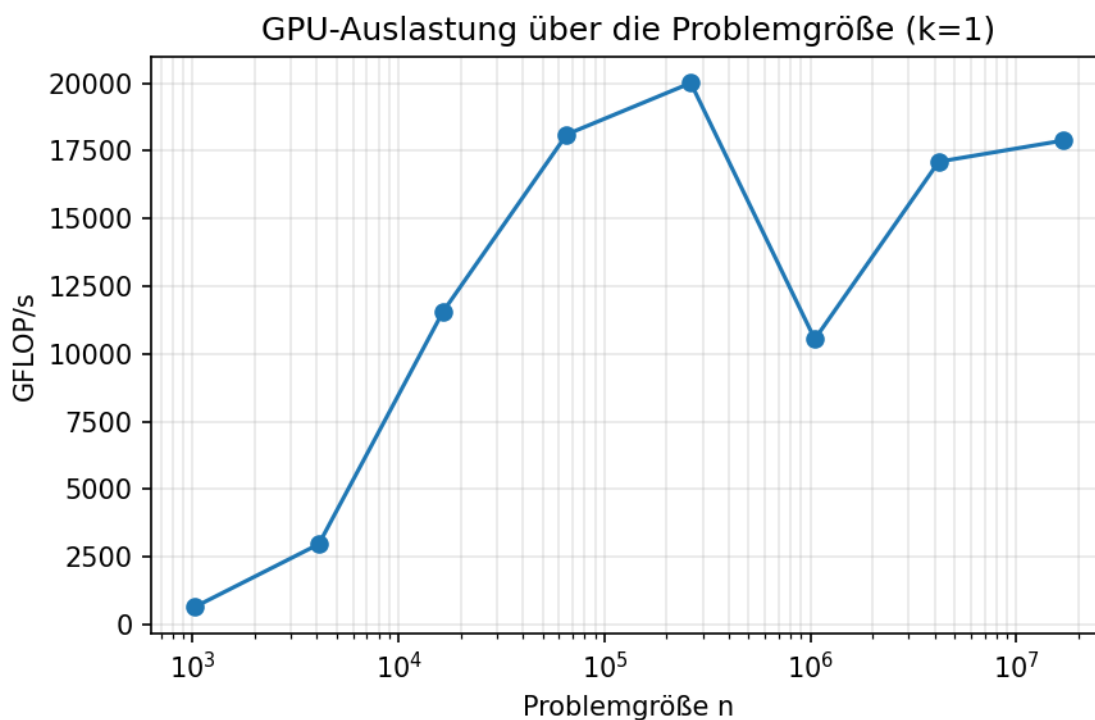


Abbildung 1: GPU-Durchsatz in GFLOP/s über die Problemgröße n (logarithmische x-Achse, $k = 1$). Sättigung ab ca. 260.000 Threads bei rund 20 TFLOP/s.

Wirkung der Warp-Divergenz

Anschließend habe ich bei fester Problemgröße ($n = 16,7$ Mio.) den Divergenzgrad k systematisch verdoppelt. Tabelle 1 fasst die Messwerte zusammen, Abbildung 2 stellt sie grafisch dar.

k (Pfade pro Warp)	GPU (GFLOP/s)	Einbruch ggü. k=1	CPU skalar (GFLOP/s)	CPU AVX2 (GFLOP/s)
1	19.049	1,0×	1,6	18,9
2	10.347	1,8×	1,6	9,4
4	5.196	3,7×	1,6	4,7
8	2.666	7,1×	1,6	2,3
16	1.341	14,2×	1,6	—
32	686	27,8×	1,6	—

Tabelle 1: Durchsatz in Abhängigkeit vom Divergenzgrad k für GPU, skalare CPU und CPU mit AVX2. Bei AVX2 ist k auf 8 begrenzt, da der Vektor nur 8 Lanes besitzt.

Der GPU-Durchsatz fällt fast exakt mit $1/k$: jede Verdopplung des Divergenzgrads halbiert die Leistung, bei $k = 32$ bleibt nur noch ein 27,8-tel der Ausgangsleistung übrig. Dass der Faktor nicht exakt 32 beträgt, liegt an einem kleinen konstanten Arbeitsanteil (Indexberechnung, das eine Load/Store-Paar), der nicht mitserialisiert wird. In Abbildung 2 liegt die GPU-Kurve deshalb praktisch deckungsgleich auf der gestrichelten Ideallinie $1/k$.

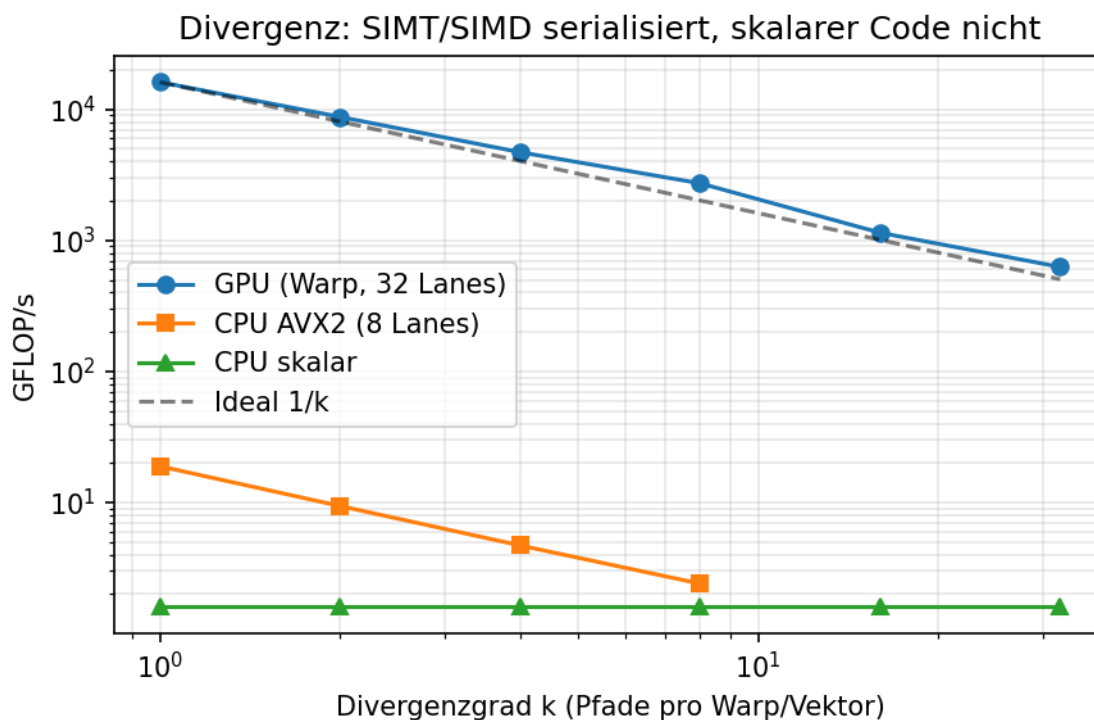


Abbildung 2: Durchsatz über den Divergenzgrad k (doppelt-logarithmisch): GPU (32 Lanes), CPU mit AVX2 (8 Lanes) und skalare CPU im Vergleich zur Ideallinie $1/k$.

Warum serialisiert ein Warp und warum ist die CPU immun?

Die Erklärung liegt im SIMT-Modell: Ein Warp führt (logisch) einen gemeinsamen Instruktionsstrom aus. Stoßen die Lanes auf eine datenabhängige Verzweigung, kann die Hardware nicht 32 verschiedene Programmzähler verfolgen. Stattdessen werden die divergenten Pfade nacheinander

ausgeführt, wobei eine aktive Maske festhält, welche Lanes am aktuellen Pfad beteiligt sind. Alle übrigen Lanes sind währenddessen deaktiviert und leisten keine Arbeit. Bei k Pfaden rechnen also stets nur $32/k$ Lanes, während die Laufzeit auf das k -Fache steigt. Meine Messung bestätigt genau dieses Modell.

Auf der CPU ist dieselbe Verzweigung dagegen nahezu kostenlos: der skalare Durchsatz bleibt über alle k konstant bei 1,6 GFLOP/s. Ein CPU-Kern besitzt einen eigenen, unabhängigen Kontrollfluss und einen Branch Predictor (Sprungvorhersage), der das regelmäßige Muster $i \% k$ nach wenigen Iterationen perfekt vorhersagt. Die Pipeline muss praktisch nie verworfen werden. Hier wird der SIMD-Unterschied zu einer „normalen“ CPU sichtbar, den das Aufgabenblatt herausarbeiten will: nicht die Verzweigung an sich ist teuer, sondern die Verzweigung innerhalb einer im Gleichschritt laufenden Gruppe.

Erweiterung: Divergenz ist eine SIMD-Eigenschaft

Der Vergleich „GPU bricht ein, CPU nicht“ greift meiner Meinung nach etwas zu kurz, denn auch moderne CPUs rechnen intern im Gleichschritt, nämlich in ihren SIMD-Vektoreinheiten (Single Instruction, Multiple Data). Um das zu zeigen, habe ich dieselbe Verzweigung zusätzlich mit AVX2-Intrinsics implementiert. Ein AVX2-Vektor verarbeitet 8 Gleitkommazahlen (Lanes) mit einer Instruktion und kann genau wie ein Warp nicht pro Lane verzweigen. Meine Implementierung rechnet deshalb alle k Pfade vollständig durch und übernimmt anschließend per Blend-Operation (einer Misch-Instruktion mit Maske) nur die Lanes des jeweils passenden Pfads. Diese Maske ist der analoge Fall zur aktiven Maske eines Warps.

Das Ergebnis in Tabelle 1 und Abbildung 2 bestätigen die Annahme: Ohne Divergenz erreicht die AVX2-Version 18,9 GFLOP/s, also rund das Zwölfwache der skalaren Version (8 Lanes multipliziert mit dem FMA-Vorteil). Mit steigendem k fällt sie exakt mit $1/k$, parallel zur GPU-Kurve und bei $k = 8$ ist der gesamte SIMD-Vorteil aufgezehrt: AVX2 und skalarer Code liegen fast gleichauf. Der relevante Gegensatz ist nicht „GPU gegen CPU“, sondern SIMD/SIMT gegen skalaren Kontrollfluss mit Sprungvorhersage. Die GPU hat mit 32 Lanes pro Warp lediglich viermal mehr zu verlieren als AVX2 mit 8 Lanes, weshalb k bei AVX2 auch nur bis 8 sinnvoll ist.

Aufgabe 2: Speicherlatenz verstecken und Bandbreite

Die zweite Aufgabe verlangt einen speicherintensiven Kernel mit geringer arithmetischer Intensität, den Vergleich dreier Zugriffsmuster auf den Gerätespeicher (coalesced, gestriped, zufällig), die Angabe der erreichten Bandbreite im Verhältnis zum Maximum des Geräts sowie den Nachweis des Latenz-Versteckens über die Occupancy. Mit arithmetischer Intensität ist das Verhältnis von Rechenoperationen zu bewegten Bytes gemeint. Mein Streaming-Kernel $b[i] = a[idx[i]] * c$ führt pro 12 bewegten Bytes nur eine einzige Multiplikation aus und ist damit eindeutig speicher- und nicht rechenlimitiert.

Der Trick in der Implementierung ist das Index-Array idx : das Zugriffsmuster wird ausschließlich über dessen Inhalt gesteuert, während der Kernel selbst für alle drei Muster völlig identisch bleibt. Für das coalesced-Muster (zusammenhängend, Stride 1) gilt $idx[i] = i$. Benachbarte Threads lesen benachbarte Adressen, sodass die Speicherzugriffe eines Warps zu wenigen breiten Transaktionen zusammengefasst („koalesziert“) werden können. Beim Stride- k -Muster liegen die gelesenen

Adressen benachbarter Threads k Elemente auseinander, und beim Gather-Muster ist idx eine zufällige Permutation. Jeder Thread greift an eine praktisch beliebige Stelle. Die bewegte Datenmenge beträgt in allen Fällen 12 Byte pro Element (4 Byte Index lesen, 4 Byte aus a lesen, 4 Byte nach b schreiben). Das Index-Array kostet zwar zusätzliche Bandbreite, macht die drei Muster aber exakt vergleichbar. Wichtig für die Interpretation ist, dass nur der Zugriff auf a variiert, während Index-Lesen und Schreiben stets coalesced bleiben.

Die theoretische Spitzenbandbreite frage ich zur Laufzeit über die CUDA-Attribut-API ab (im CUDA Toolkit 13 wurden die entsprechenden Felder aus der Geräteeigenschafts-Struktur entfernt, was ich beim ersten Kompilieren erst herausfinden musste) und gebe jede Messung sowohl absolut in GB/s als auch als Prozentsatz dieses Peaks an.

Meine ersten Messläufe lieferten ein physikalisch unsinniges Ergebnis: das Stride-2-Muster war angeblich schneller als das coalesced-Muster und derselbe coalesced-Kernel erreichte an späterer Stelle im Programm fast die doppelte Bandbreite wie in der ersten Messung. Die Ursache war die Taktsteuerung der GPU: die Karte fährt ihre Boost-Taktraten erst nach einigen Kernel-Starts hoch, ein einzelner Warmup-Lauf reichte dafür nicht aus. Die allererste Messung war dadurch systematisch zu langsam. Nach Umstellung auf drei Warmup-Läufe und Mittelung über fünf Wiederholungen waren die Ergebnisse konsistent und reproduzierbar.

Zugriffsmuster und effektive Bandbreite

Zugriffsmuster	Bandbreite (GB/s)	Anteil am Peak (448 GB/s)
coalesced (Stride 1)	389,8	87 %
Stride 2	307,6	69 %
Stride 4	194,7	43 %
Stride 8	116,3	26 %
Stride 16	114,0	25 %
Stride 32	119,3	27 %
Gather (zufällig)	49,2	11 %

Tabelle 2: Effektive Bandbreite der drei Zugriffsmuster bei Blockgröße 256, jeweils absolut und relativ zur theoretischen Spitzenbandbreite der RTX 3070.

Das coalesced-Muster erreicht 389,8 GB/s und damit 87 % des theoretischen Peaks, was für einen realen Streaming-Kernel ein sehr guter Wert, der zugleich belegt, dass der Kernel tatsächlich bandbreitenlimitiert ist. Mit wachsendem Stride fällt die Bandbreite zunächst deutlich (69 % bei Stride 2, 43 % bei Stride 4), verharrt ab Stride 8 aber auf einem Plateau um 115–120 GB/s. Dieses Plateau lässt sich gut erklären: Der Gerätespeicher wird in Transaktionen von 32 Byte angesprochen. Ab Stride 8 liegen die von einem Warp gelesenen 4-Byte-Elemente jeweils 32 Byte oder weiter auseinander, sodass jedes Element eine eigene Transaktion benötigt. Die Verschwendung beim Lesen von a ist damit bereits maximal und kann durch größere Strides nicht weiterwachsen. Dass die Bandbreite nicht auf ein Achtel des coalesced-Werts abstürzt, liegt an den weiterhin coalesced ablaufenden Index- und Schreibzugriffen, die zwei Drittel der gezählten Datenmenge ausmachen.

Das Gather-Muster bildet mit 49,2 GB/s (11 % des Peaks) das Schlusslicht: bei zufälligen Adressen transportiert nahezu jede 32-Byte-Transaktion nur 4 nützliche Bytes und auch der L2-Cache kann bei

einer zufälligen Permutation über 64 MB Daten nichts retten. Zwischen dem besten und dem schlechtesten Muster liegt damit ein Faktor von etwa 8 bei identischem Kernel und identischer Datenmenge. Abbildung 3 stellt die Ergebnisse als Balkendiagramm mit eingezeichneter Peak-Linie dar.

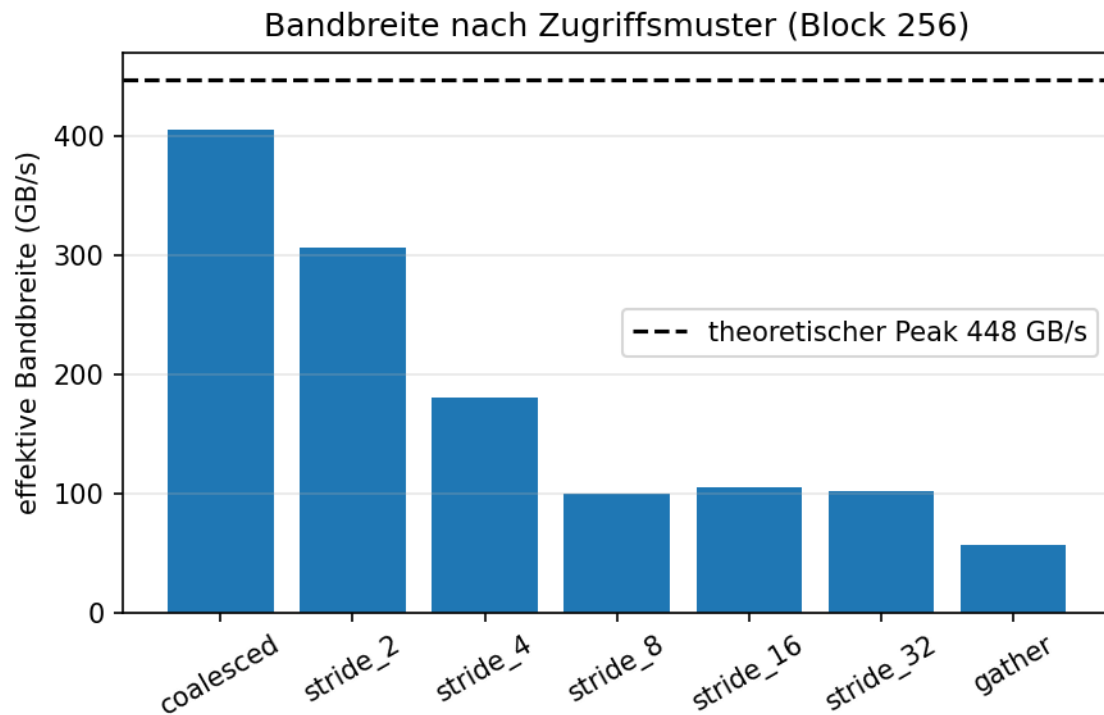


Abbildung 3: Effektive Bandbreite nach Zugriffsmuster (Blockgröße 256). Die gestrichelte Linie markiert die theoretische Spitzenbandbreite von 448 GB/s.

Ergebnisse: Latency Hiding über die Occupancy

Der zweite Teil der Aufgabe betrifft das Verstecken der Speicherlatenz. Jeder Zugriff auf den Gerätespeicher (DRAM) dauert mehrere hundert Nanosekunden. Würde ein Warp einfach warten, läge der SM die meiste Zeit brach. Stattdessen tauscht der Warp-Scheduler wartende Warps ohne Umschaltkosten gegen rechenbereite aus. Die Latenz wird also nicht vermieden, sondern durch andere Arbeit überdeckt. Wie gut das funktioniert, hängt davon ab, wie viele Warps gleichzeitig auf einem SM resident sind. Genau das misst die Occupancy: das Verhältnis aus aktiven Warps zu den maximal möglichen aktiven Warps pro SM (bei der RTX 3070 sind das 48). Ich habe die Occupancy über die Blockgröße variiert und mit der offiziellen CUDA-Occupancy-API berechnet.

Blockgröße (Threads)	Occupancy	Bandbreite (GB/s)
32	0,33	221,4
64	0,67	287,5
128	1,00	403,6
256	1,00	406,6
512	1,00	407,6
1024	0,67	357,5

Tabelle 3: Effektive Bandbreite des coalesced-Kernels in Abhängigkeit von Blockgröße und daraus resultierender Occupancy.

Die Messung (Tabelle 3, Abbildung 4) zeigt den erwarteten Zusammenhang: Bei einer Occupancy von 0,33 stehen zu wenige Warps bereit, um die Speicher-Stalls zu überbrücken. Die Bandbreite bleibt bei 221 GB/s hängen, fast die Hälfte der Speicherzeit wird also verschenkt. Mit 0,67 steigt sie auf 288 GB/s und bei voller Occupancy sättigt sie bei rund 405 GB/s: ab hier ist nicht mehr die Anzahl der Warps der Engpass, sondern der Speicher selbst.

Ein Detail hat mich zunächst überrascht: die Blockgröße 1024 erreicht trotz derselben Occupancy wie Blockgröße 64 (jeweils 0,67) mit 357,5 GB/s einen deutlich anderen Wert. Die Erklärung liegt in der Granularität: bei 1024er-Blöcken passt nur ein einziger Block auf einen SM (maximal 1536 Threads pro SM auf Ampere). Blöcke werden aber als Ganzes zugeteilt und beendet. Startet oder endet ein solcher Riesenblock, liegt kurzzeitig ein großer Teil des SMs brach und die Warps eines einzelnen Blocks laufen zudem nicht perfekt gleichmäßig aus. Die Occupancy allein ist also kein hinreichendes Kriterium für gute Leistung; die Blockgröße sollte dem Scheduler genügend Spielraum lassen.

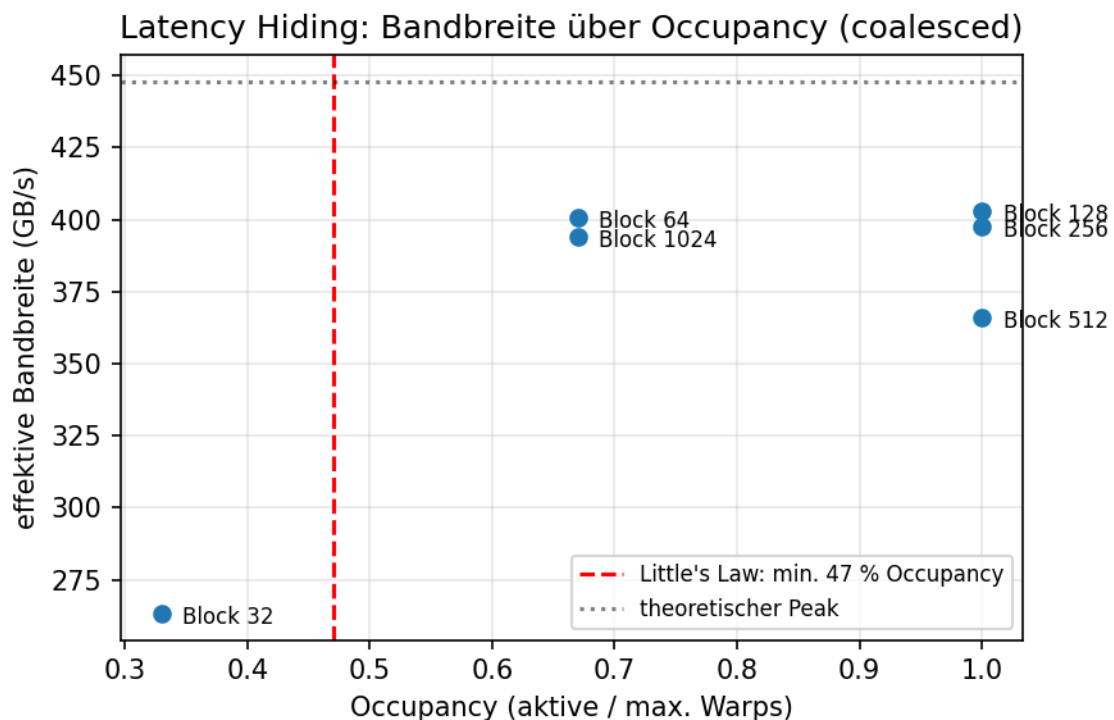


Abbildung 4: Effektive Bandbreite über die Occupancy (coalesced-Muster), annotiert mit den zugehörigen Blockgrößen. Die rote Linie zeigt die aus Little's Law vorhergesagte Mindest-Occupancy von 47 %.

Erweiterung: Vorhersage der nötigen Parallelität mit Little's Law

Die Occupancy-Messung zeigt, dass genügend Warps nötig sind, aber sie beantwortet nicht, wie viele es sein müssen. Das lässt sich mit Little's Law aus der Warteschlangentheorie herleiten: um einen Durchsatz B bei einer Latenz L aufrechtzuerhalten, müssen im Mittel $B \times L$ Bytes gleichzeitig „unterwegs“ (in flight) sein [1]. Dazu brauchte ich zunächst die rohe DRAM-Latenz meiner Karte.

Gemessen habe ich sie per Pointer Chasing: ein einzelner Thread hangelt sich durch einen zufälligen Zyklus über ein 128-MB-Array (bewusst deutlich größer als der 4-MB-L2-Cache, damit wirklich der DRAM und nicht der Cache gemessen wird). Da jeder Zugriff vom Ergebnis des vorherigen abhängt, kann nichts überlappt oder vorhergesagt werden. Man sieht die nackte Latenz, die im normalen Betrieb versteckt wird. Das Ergebnis: 299 ns pro Zugriff.

Daraus folgt: $448 \text{ GB/s} \times 299 \text{ ns} \approx 134 \text{ kB}$ müssen permanent in flight sein. Unter der vereinfachenden Annahme, dass ein Warp beim coalesced Streaming etwa eine ausstehende 128-Byte-Ladeanfrage hält ($32 \text{ Threads} \times 4 \text{ Byte}$), entspricht das rund 1.050 Warps auf der ganzen GPU bzw. etwa 23 Warps pro SM und einer Mindest-Occupancy von rund 47 %. Diese Vorhersage passt bemerkenswert gut zur unabhängig davon gemessenen Occupancy-Kurve: bei 33 % ist die Bandbreite klar unterversorgt, oberhalb von rund 50 % nähert sie sich der Sättigung (rote Linie in Abbildung 4). Die 128-Byte-Annahme ist zugegebenermaßen eine Vereinfachung. Das Modell liefert demnach die Größenordnung und keine Nachkommastellen. Aber genau darin liegt für mich der Wert dieser Erweiterung: die Occupancy-Kurve wird nicht nur gezeigt, sondern aus zwei unabhängigen Messgrößen (Latenz und Bandbreite) quantitativ hergeleitet.

Damit lässt sich die Erklärungsfrage des Aufgabenblatts beantworten. CPU und GPU verbergen Speicherlatenz auf grundverschiedene Weise. Die CPU setzt auf eine tiefe Cache-Hierarchie und Prefetching: häufig genutzte Daten liegen in schnellen Caches und bei sequenziellen Zugriffen lädt der Prefetcher die nächsten Cache-Zeilen spekulativ voraus. Sequenzielle Muster sind dadurch sehr schnell, zufällige Muster durch Cache-Misses langsam, aber ein einzelner Thread kommt gut voran. Die GPU dagegen versteckt Latenz über massive Parallelität: wartende Warps werden einfach gegen rechenbereite ausgetauscht. Ihre Caches sind pro Thread winzig und könnten die Arbeitsmengen zehntausender Threads ohnehin nicht halten. Sie dienen eher der Bündelung von Zugriffen als der Wiederverwendung von Daten.

Aus diesem Unterschied ergeben sich unmittelbare Konsequenzen für die Datenanordnung in GPU-Anwendungen. Entscheidend ist nicht Cache-Freundlichkeit im CPU-Sinn, sondern Coalescing: die 32 Lanes eines Warps müssen gemeinsam zusammenhängende Adressen treffen. Praktisch bedeutet das erstens Structure of Arrays statt Array of Structures. Liegen etwa die x-Koordinaten aller Partikel hintereinander im Speicher, liest ein Warp sie in einer Transaktion. Liegt jedes Partikel als Struktur mit allen Feldern am Stück, entsteht automatisch ein Stride-Muster. Zweitens müssen zusammen verarbeitete Daten auch zusammenhängend angeordnet sein, und drittens braucht der Kernel genügend Parallelität (ausreichend residente Warps oder mehrere unabhängige Zugriffe pro Thread), damit das Latency Hiding überhaupt greifen kann. Meine Messungen quantifizieren, was auf dem Spiel steht: zwischen gutem und schlechtem Zugriffsmuster liegt Faktor 8, zwischen

ausreichender und unzureichender Occupancy fast Faktor 2. Im Zweifel lohnt es sich daher mehr, die Daten vor dem Kernel einmal umzusortieren, als am Kernel selbst zu optimieren.

Fazit

Beide Experimente machen denselben Grundgedanken von zwei Seiten sichtbar: die GPU erkaufte ihren enormen Durchsatz durch Gleichschritt und Masse. Aufgabe 1 zeigt die Kehrseite des Gleichschritts: divergente Verzweigungen innerhalb eines Warps werden serialisiert und kosten bis zu Faktor 28, während dieselbe Verzweigung auf einem skalaren CPU-Kern dank Sprungvorhersage kostenlos ist. Die AVX2-Erweiterung belegt, dass dies eine Eigenschaft aller SIMD-Ausführung ist. Aufgabe 2 zeigt die Kehrseite der Masse: die GPU braucht das richtige Zugriffsmuster (Coalescing, Faktor 8) und genügend residente Warps (Occupancy), um ihre Bandbreite abzurufen. Mit Little's Law lässt sich der nötige Parallelitätsgrad sogar vorhersagen. Wer für die GPU programmiert, muss beides mitdenken: einheitlichen Kontrollfluss innerhalb der Warps und eine Datenanordnung, die zum kollektiven Zugriffsmuster passt.

Nutzung von KI-Werkzeugen

Zur Erstellung und Programmierung der CUDA-C++-Quellcodedateien (.cu) wurde als Hilfsmittel von Anthropic Claude Fable 5 (Mittlerer Aufwand) verwendet. Ebenso wurde dieser teilweise zur Berechnung und Interpretation der Messergebnisse mit Bezug zu Little's Law verwendet.

Quellen

[1] Gustafson, J.L. (2011). Little's Law. In: Padua, D. (eds) Encyclopedia of Parallel Computing. Springer, Boston, MA. https://doi.org/10.1007/978-0-387-09766-4_79